FUNÇÕES, E DE C PARA MIPS · ENCONTRO 5 · LIGA PROTOFPGA

Gabarito

Parte A — Atividade em aula

Item	Resposta
1	Três: no fim de `soma_vetor`, de `maior_vetor` e de `imprime`. As três têm o mesmo código, `0x03e00008`, porque o destino não está na instrução — está no registrador `$ra`, que muda a cada chamada.
2	`$ra` = **`0x00400010`** e `pc` = **`0x00400050`**. O `$ra` é o endereço da instrução seguinte ao `jal` (que está em `0x0040000c`): PC + 4. O `pc` é o endereço do rótulo `soma_vetor`, a primeira instrução da função.
3	`$v0` = `0x00000029` = 41: a soma do vetor, o resultado da função. Depois do `jr $ra`, o `pc` vale **`0x00400010`** — o valor que estava em `$ra`, a instrução `move $s0, $v0` do `main`.
4	Um parâmetro: a função lê `$a0` (no `slt`) e nunca usa `$a1`. Devolve o que está em `$v0`: a soma acumulada no laço.
5	Ver abaixo.
6	`$s0` = misterio(7) = **16** (1 + 3 + 5 + 7); `$s1` = misterio(10) = **25** (1 + 3 + 5 + 7 + 9).
7	`$sp` = **`0x7FFFEFDC`**. O `jal fact` executou **4** vezes: uma no `main` e três de dentro de `fact` (para n = 2, 1 e 0).
8	Ver a tabela abaixo.
9	`Soma = 8` e `Maior = 9`.
10	Entre guardar a soma em `$t0` e imprimi-la, o `main` chamou `maior_vetor`, que usa `$t0` como o contador `i`. Quando ela voltou, `$t0` valia 8 — o valor final de `i`, igual a `n`. A soma foi apagada sem erro nenhum. Os `$t` não sobrevivem a um `jal`; por isso o original usava `$s0`.
11	O `fact(5) = …` nunca aparece: o MARS para com *Runtime exception at 0x00400074: arithmetic overflow*. Sem guardar o `$ra`, todo `jr $ra` volta para a instrução depois do `jal fact` de dentro, e o programa nunca mais sai de `fact`; a cada volta, `addi $sp, $sp, 8` sobe o `$sp`, até ele passar de `0x7FFFFFFF` e a soma transbordar. Frase exemplo: “Sem guardar o endereço de volta, a função esquece para onde voltar e fica presa dentro de si mesma.”

Item 5 — uma resposta possível (o `for` também pode ser escrito como `while`):

```
int soma_impares(int n) {
    int s = 0;
    for (int i = 1; i <= n; i = i + 2)
        s = s + i;
    return s;
}
```

Pistas usadas: `li $v0, 0` e `li $t0, 1` são as inicializações; o teste “`n < i` → sai” é `i <= n` invertido; o incremento é `addi $t0, $t0, 2`. Outros nomes bons: `soma_dos_impares_ate`, `soma_impares_ate_n`.

Item 8 — a pilha no ponto de parada, com n = 3:

Endereço	Conteúdo	$ra ou n?	De qual chamada?
0x7FFFEFF8	0x0040000c	$ra	fact(3)
0x7FFFEFF4	0x00000003	n	fact(3)
0x7FFFEFF0	0x00400070	$ra	fact(2)
0x7FFFEFEC	0x00000002	n	fact(2)
0x7FFFEFE8	0x00400070	$ra	fact(1)
0x7FFFEFE4	0x00000001	n	fact(1)
0x7FFFEFE0	0x00400070	$ra	fact(0)
0x7FFFEFDC	0x00000000	n	fact(0)

O primeiro `$ra` aponta para o `main` (a instrução depois de `jal fact`, em `0x00400008`). Os outros três são iguais: todos voltam para a instrução depois do `jal fact` de dentro de `fact`, em `0x0040006c`. Na janela Data Segment, cada linha mostra oito palavras, da esquerda para a direita, do endereço mais baixo para o mais alto — na ordem inversa desta tabela. O endereço de cada valor é o da linha mais o deslocamento da coluna (+0, +4, +8…).

Sobre os códigos e endereços deste gabarito

Todos os programas, códigos, endereços e saídas foram conferidos no MARS 4.5. O MARS mostra as letras hexadecimais em minúsculas (`0x0040000c`); é o mesmo número que `0x0040000C`.

Parte B — Desafio da semana

Uma solução possível (existem outras igualmente corretas):

```
        .data
vetor:  .word  7, 3, 9, 2, 5, 8, 1, 6
msg1:   .asciiz "Maiores que 5: "
msg2:   .asciiz "Maiores que o limite: "

        .text
        .globl main
main:
        la   $a0, vetor
        li   $a1, 8
        li   $a2, 5             # 3º argumento: limite
        jal  conta_maiores
        move $s0, $v0           # guarda: a próxima chamada usa $v0

        li   $v0, 5             # lê o limite do teclado
        syscall
        la   $a0, vetor
        li   $a1, 8
        move $a2, $v0
        jal  conta_maiores
        move $s1, $v0

        la   $a0, msg1
        move $a1, $s0
        jal  imprime
        la   $a0, msg2
        move $a1, $s1
        jal  imprime
        li   $v0, 10
        syscall

# conta_maiores(endereço em $a0, n em $a1, limite em $a2) -> $v0
conta_maiores:
        li   $v0, 0             # contador
        li   $t0, 0             # i
cm_laco: slt  $t1, $t0, $a1
        beq  $t1, $zero, cm_fim
        lw   $t2, 0($a0)
        slt  $t1, $a2, $t2      # limite < v[i] ?
        beq  $t1, $zero, cm_pula
        addi $v0, $v0, 1
cm_pula: addi $t0, $t0, 1
        addi $a0, $a0, 4
        j    cm_laco
cm_fim:  jr   $ra

# imprime: a mesma do aula5_funcoes.asm
```

Conferido no MARS: com o limite lido valendo 0, o Run I/O mostra `Maiores que 5: 4` e `Maiores que o limite: 8`; com 9, `Maiores que 5: 4` e `Maiores que o limite: 0`.

Pergunta extra. O primeiro resultado precisa ir para um `$s` — aqui, `$s0`. Ele precisa sobreviver a três coisas: ao `syscall` de leitura (que escreve em `$v0`), à segunda chamada de `conta_maiores` (que

usa `$v0`, `$t0`, `$t1`, `$t2` e altera `$a0`) e à primeira chamada de `imprime`. Num `$t`, poderia funcionar por sorte; num `$v0`, seria apagado logo no `li $v0, 5`.

Os dois erros mais comuns: guardar o limite lido direto em `$a2` **antes** de preparar `$a0` e `$a1` — funciona, mas é fácil confundir —; e esquecer que o número lido chega em `$v0` e copiar `$a0` em vez de `$v0` para `$a2`.

Parte C — Exercícios de revisão

1. Primeiro guarda em `$ra` o endereço da instrução seguinte (PC + 4); depois salta para o rótulo (PC ← endereço do rótulo).

2. Porque `j` não anota a volta: `$ra` continua com o valor que tinha antes. O `jr $ra` da função voltaria para esse lugar qualquer. Conferido no MARS: trocando o primeiro `jal soma_vetor` por `j soma_vetor`, o programa para com *invalid program counter value: 0x00000000* — o `$ra` ainda valia zero.

3. `$a0` = endereço de `v`, `$a1` = 8, `$a2` = `lim`. Logo depois da chamada, `r` está em `$v0`.

4. **$s2**: sim — a função chamada tem de devolvê-lo intacto. **$sp**: sim — o epílogo o devolve ao valor de antes. **$t3**: não — a função pode usá-lo à vontade. **$a0**: não — é registrador de argumento e a função pode alterá-lo, como `soma_vetor` faz. **$v0**: não — agora contém o resultado. **$ra**: não — o próprio `jal` escreveu nele.

5. Porque o `jal` de dentro sobrescreve o `$ra`, apagando o endereço de volta da própria função; sem guardá-lo antes, o `jr $ra` final volta para o lugar errado. Uma folha não faz nenhum `jal`, então o `$ra` que recebeu continua intacto até o `jr`.

6. `$sp` = `0x7FFFEFF0` (`0x7FFFEFFC − 12`; 12 = `0xC`). Foram reservadas 3 palavras (12 ÷ 4). O epílogo precisa executar `addi $sp, $sp, 12`, para o `$sp` voltar exatamente a `0x7FFFEFFC`.

7. Uma resposta possível:

```
int conta_negativos(int v[], int n) {
    int c = 0;
    for (int i = 0; i < n; i++)
        if (v[i] < 0)
            c = c + 1;
    return c;
}
```

Com `v = {4, −2, 7, −9, 0, −1}`, devolve **3** (−2, −9 e −1; o zero não conta). Conferido no MARS. Pistas: `slt $t1, $t2, $zero` pergunta se `v[i] < 0`, e o `beq` pula o `addi` quando não é.

8. Uma solução:

```
main:   li   $a0, 3
        jal  dobro              # dobro(3)
        move $a0, $v0           # o resultado vira o argumento
        jal  dobro              # dobro(dobro(3))
        move $s0, $v0           # y
        li   $v0, 1
        move $a0, $s0
        syscall
        li   $v0, 10
        syscall

dobro:  add  $v0, $a0, $a0
        jr   $ra
```

`dobro` não precisa de quadro: é uma folha e não usa nenhum `$s`. y = **12**, conferido no MARS. Repare na chamada aninhada `dobro(dobro(3))`: o C a avalia de dentro para fora, e o resultado de uma chamada, em `$v0`, vira o argumento da outra, em `$a0`.

9. (a) **5** vezes: uma no `main` e quatro de dentro de `fact` (para n = 3, 2, 1 e 0). (b) **5** quadros, no fundo da recursão: os de `fact(4)`, `fact(3)`, `fact(2)`, `fact(1)` e `fact(0)`. (c) `0x7FFFEFFC − 5 × 8 = 0x7FFFEFFC − 40 =` **`0x7FFFEFD4`** (40 = `0x28`).

10. Em binário, `0000 1100 0001 0000 0000 0000 0001 0100`. Os 6 primeiros bits:

op	endereço (26 bits)
`000011`	**`00 0001 0000 0000 0000 0001 0100`**
3 → jal	0x0100014

É um `jal`. O destino é `0x0100014 × 4 = 0x00400050`, o endereço de `fact`. As **duas** chamadas `jal fact` do arquivo — a do `main` e a de dentro de `fact` — têm exatamente esse código, porque saltam para o mesmo lugar.

11. Uma solução:

```
        .data
msg:    .asciiz "Soma = "
        .text
        .globl main
main:   li   $v0, 5
        syscall
        move $a0, $v0           # n
        jal  soma
        move $s0, $v0
        li   $v0, 4
        la   $a0, msg
        syscall
        li   $v0, 1
        move $a0, $s0
        syscall
        li   $v0, 10
        syscall

# soma(n) = n + soma(n - 1);  soma(0) = 0
soma:
        addi $sp, $sp, -8
        sw   $ra, 4($sp)
        sw   $a0, 0($sp)
        bne  $a0, $zero, s_geral
        li   $v0, 0             # soma(0) = 0
        j    s_sai
s_geral:
        addi $a0, $a0, -1
        jal  soma               # $v0 = soma(n - 1)
        lw   $a0, 0($sp)
        add  $v0, $a0, $v0      # n + soma(n - 1)
s_sai:  lw   $ra, 4($sp)
        addi $sp, $sp, 8
        jr   $ra
```

Conferido no MARS: 10 → `Soma = 55`; 100 → `Soma = 5050`; 0 → `Soma = 0`. É o `fact` com `add` no lugar de `mul` e 0 no lugar de 1 no caso base.

Memória. A versão com laço do encontro 4 não usa a pilha: quatro registradores bastam, qualquer que seja n. A versão recursiva abre um quadro de 8 bytes por chamada e, para n = 100, chega a ter **101** quadros ao mesmo tempo (de `soma(100)` a `soma(0)`): 808 bytes de pilha. A recursão é mais parecida com a definição matemática; o laço é mais econômico. Os dois dão o mesmo resultado.